

Andrew Cotaj

(203)-561-7076 | ajcotaj@gmail.com | [linkedin.com/in/andrew-cotaj](https://www.linkedin.com/in/andrew-cotaj) | adrewc2026.github.io/Andrew-Cotaj

EXECUTIVE SUMMARY

Notre Dame Computer Science graduate and quantitative systems engineer operating at the intersection of high-frequency trading, machine learning, and trustworthy AI infrastructure. Proven builder of low-latency trading systems, including a compile-time-verified OCaml matching engine, a multithreaded exchange-connected trading bot, and an end-to-end market-impact research framework validated on 50M+ ticks/session of TAQ and crypto microstructure data. Delivered a 107% throughput increase architecting AI-driven data pipelines at ASML and pioneered agentic safety research (model serialization security, adversarial attack detection, memory-safe Rust) at Notre Dame's Security and Software Engineering Lab. I combine rigorous measurement discipline (every performance claim benchmarked, every backtest controlled) with production-grade engineering across Python, C++, OCaml, and Rust, purpose-built for environments where latency, correctness, and risk are mission-critical.

EDUCATION

University of Notre Dame | Notre Dame, IN **Graduated May 2026**

Bachelor of Science, Computer Science | Minor: Engineering Corporate Practice
GPA: 3.76/4.0 | Dean's List (Spring 2023, Fall 2024)

Hopkins School | New Haven, CT **Graduated Jun. 2022**

GPA: 4.59 | SAT: 1520 | Quintile 1/5 | Valedictorian Finalist | AP Scholar with Distinction

WORK EXPERIENCE

ASML | Wilton, CT **May – Aug. 2025**

Software Engineering Intern

- Architected an end-to-end Python performance-evaluation platform leveraging the GitHub API to collect, filter, and organize code-review telemetry; deployed as an automated workflow pipeline, backend service, and web application, resulting in a 107% increase in team throughput
- Operationalized high-usage AI backend services within semiconductor manufacturing; conducted cost-benefit analysis for company-wide deployment, prioritizing low-latency integration and systemic reliability with existing backend systems
- Engineered scalable DevOps workflows bridging raw code-review data to actionable business intelligence, proving the economic viability of internal GenAI-driven performance metrics; tool on track for company-wide implementation

Security and Software Engineering Lab | University of Notre Dame **Aug. 2024 – May 2026**

Research Assistant: AI-Based Software Security & Agentic Trust (4 semesters, advised by Prof. Joanna C. S. Santos)

- Pioneered research in agentic safety by analyzing model serialization vulnerabilities in the Hugging Face ecosystem (pickle, torch.save); documented the transition to safetensors to prevent object-injection attacks in production ML environments
- Co-developed TITAN, a hybrid quantum-ML framework using Projected Quantum Kernels (Qiskit) to generate quantum-enhanced features and train SVM classifiers for adversarial attack detection in NLP models; constructed adversarial datasets and benchmarked against state-of-the-art detectors
- Conducted a systematic survey of C-to-Rust transpilation techniques, evaluating tools, semantic preservation, and memory-safety guarantees for deploying trustworthy, high-stakes AI infrastructure (targeted for publication)
- Performed empirical analysis of Quantum Programming Languages (QPLs), assessing language features, adoption trends, and applicability to hybrid quantum-classical workloads

QUANTITATIVE FINANCE & HFT PROJECTS

OxCaml Limit-Order-Book Matching Engine **Dec. 2025 – Jul. 2026**

Independent | OCaml/OxCaml (Jane Street's production compiler fork), GADTs, qcheck, GitHub Actions CI

- Engineered an exchange-side matching engine with price-time priority and five order types (limit, market, IOC, FOK, post-only), modeled as a pure, total state-transition function enabling deterministic replay and invariant-preserving execution at the resting (maker) price
- Made illegal states unrepresentable at compile time: GADT-indexed order lifecycle forbids re-filling or reading remaining quantity on filled orders; abstract types with validating smart constructors guarantee non-negative prices/positive quantities; polymorphic-variant rows make resting market orders and invalid time-in-force combinations unrepresentable
- Eliminated GC pressure on the matching hot path using OCaml's local mode with stack-allocated fills over cache-friendly flat parallel arrays: measured 32.0 → 0.0 bytes/match and 7 → 0 minor collections over 2,000,000 makers
- Achieved data-race-freedom by construction via OCaml portability/contention modes and immutable_data kinds: built a multi-domain order-entry gateway sustaining ~2.95M orders/sec (4 producers → 1 matcher), with a checked-in compile-error proof that sharing a mutable book across domains cannot compile
- Validated correctness with a qcheck property-based suite asserting five invariants (share conservation, no crossed book, FIFO within level, price-time priority, deterministic replay) across thousands of randomized command streams; 7/7 properties and 16/16 regression cases green in two-job CI against the real OCaml compiler
- Documented and audited every type-system escape hatch (4 casts with soundness justifications) and published reproducible benchmarks, measuring every claim rather than asserting it

Market-Impact & Optimal-Execution Research Framework

Feb. 2026 – Jul. 2026

Independent | Python (pandas, NumPy, LightGBM, PyArrow/Parquet), Binance microstructure data, 57 pytest cases, CI

- Built a config-driven, 3,500-LOC research library with a 10-stage idempotent Parquet-backed pipeline answering the two questions every trading desk pays for: how much does size move the market (impact), and what schedule executes a parent order at least cost (optimal execution)
- Implemented Lee-Ready trade-sign classification validated against the exchange's ground-truth aggressor flag on Binance futures data (94.1% accuracy), a validation equity tapes cannot provide
- Measured the square-root law of market impact: volatility-normalized pooling of 10 heterogeneous crypto sessions collapsed onto a single curve ($R^2 = 0.71$, exponent $0.46 \approx \frac{1}{2}$), empirically reproducing Kyle-Obizhaeva universality; recovered planted impact parameters on synthetic TAQ data to validate the measurement machinery (square-root fit $R^2 = 0.993$)
- Derived and validated Almgren-Chriss closed-form optimal liquidation trajectories three independent ways (boundary limits, discrete Euler-Lagrange identity, tridiagonal solve to $1e-6$); traced efficient frontiers and backtested TWAP/VWAP/Implementation-Shortfall across 1,500 Monte-Carlo paths per scenario, quantifying the risk-aversion crossover λ^* between regimes
- Decomposed permanent vs. transient impact via Bouchaud response functions (56% of crypto impact reverts within ~20s), recovered the Tóth metaorder square-root law across 692k reconstructed metaorders (exponent 0.58, $R^2 = 0.95$), and demonstrated real execution shortfall is leptokurtic (excess kurtosis ~17 vs. Gaussian)
- Designed a no-look-ahead closed-loop execution policy that cut CVaR₉₉ tail risk ~22% out-of-sample at matched expected cost, with a return-shuffle control proving the edge derives from genuine short-horizon path structure
- Reported an honest ML null result: LightGBM with engineered microstructure features did not beat the parametric square-root baseline out-of-sample under a purged, embargoed, forward-chaining split, demonstrating rigorous, oversell-free backtest methodology

High-Frequency Trading System: NDFEX Exchange Competition

Jan. 2026 – May 2026

University of Notre Dame | Python, multithreading, TCP/UDP multicast, binary wire protocols, pytest

- Architected a multithreaded trading engine with strict separation of concerns (a select()-driven TCP order-entry loop, a dedicated multicast market-data worker, and a ~2 ms strategy loop), synchronized with fine-grained locks and a clean shutdown path
- Implemented the full exchange wire protocol from scratch (little-endian binary structs for LOGIN/NEW/MODIFY/DELETE and ACK/REJECT/FILL/CLOSE) with a client-side order-lifecycle state machine (PENDING_NEW → OPEN → PARTIALLY_FILLED → FILLED/CANCELED)
- Built a two-layer risk system: a generic gate (per-order quantity, per-side exposure, token-bucket rate limiting, price bands, unacked-order caps) plus a competition controller enforcing hard position limits and an aggregate

PnL floor with a danger-threshold state machine, with no strategy or manual path able to bypass the gate, verified by integration tests

- Solved the order-book cold-start problem by buffering the live multicast feed during snapshot replay and detecting the exact replay→live handoff via sequence-number matching, avoiding both gaps and double-applied updates; maintained $O(\log n)$ book updates with cached $O(1)$ BBO
- Killed an unprofitable strategy on the data: measured that an 11-leg ETF arbitrage's ~120-tick crossing cost exceeded its mean reversion and disabled it: cost/edge discipline over activity
- Modeled ETF creation/redemption against a REST service with typed error handling and retry/backoff; wrote 8 pytest modules with deterministic fakes covering risk limits, breach-penalty clamping, reconnect crystallization, and end-to-end integration

S&P 500 ETF Arbitrage & iNAV Tracking Research

Jan. 2026 – May 2026

University of Notre Dame | Python (pandas, NumPy, PyArrow/Parquet), WRDS TAQ millisecond NBBO tick data

- Engineered an end-to-end pipeline reconstructing real-time intraday fair value (iNAV) of five S&P 500 ETFs (SPY, IVV, VOO, SPXL, SPXS) from share-weighted NBBO quotes of all ~500 underlying constituents, processing 50M+ quotes per trading session in columnar Parquet
- Implemented size-weighted micro-prices (order-book-imbalance weighting) for basket and ETF, plus a daily-reset leveraged-iNAV model for $\pm 3\times$ funds that correctly inverts bid/ask sides for the inverse fund
- Quantified ETF-to-iNAV mispricing distributions in basis points, detected contiguous arbitrage episodes, and measured mean-reversion speed via fitted AR(1) half-lives ($\approx 60\text{--}290$ ms across regimes); median dislocation episodes last single-digit milliseconds
- Built a latency-and-execution simulator modeling the full HFT round trip across three infrastructure tiers (co-located $\sim 22\text{ }\mu\text{s}$, realistic $\sim 98\text{ }\mu\text{s}$, remote ~ 1.6 ms) with nanosecond-benchmarked iNAV compute, per-share fees, and slippage to compute after-cost P&L
- Conducted event studies across volatility regimes, including the FOMC announcement (peak basis 27 bps) and the April 2025 tariff shock (10.9 bps) vs. calm sessions, isolating how mispricing magnitude and persistence amplify around macro catalysts
- Key finding: no arbitrage episode simultaneously cleared the >5 bps / >100 ms viability bar after fees and slippage, empirically demonstrating S&P 500 ETFs are efficiently arbitrated at sub-millisecond timescales

C++ HFT Execution & Quantitative Backtesting Engine

Jan. 2026 – May 2026

University of Notre Dame | C++, limit order book design, network programming

- Developed a low-latency trading system in C++ featuring a limit order book and order-management logic engineered for high-throughput market data
- Deployed automated market-making and arbitrage strategies in a simulated exchange, competing against rival algorithms to maximize PnL and execution speed
- Optimized performance via advanced networking for high-speed data capture, with integrated backtesting and post-trade analysis

Quantitative Deep Learning: Short-Selling & Volatility Forecasting

Aug. 2025 – Present

Independent | PyTorch, pandas, time-series modeling

- Architected a GRU neural network in PyTorch to identify overvalued securities, training on time-series data to forecast downside volatility and price depreciation for bearish risk management
- Engineered quantitative signals for alpha generation from mean reversion and statistical anomalies to detect market inefficiencies and optimize short-entry points
- Backtested and validated strategies, optimizing for Sharpe ratio and minimizing drawdown

Independent Equity Trading | Personal Portfolio Manager

Jul. 2022 – Present

- Generated a 135.71% cumulative return ($>32\%$ annualized), outperforming the S&P 500 over the same period
- Executed discretionary trades combining fundamental and technical analysis under strict risk-reward ratios to manage volatility and drawdown

AI / MACHINE LEARNING PROJECTS

MiniGPT: Transformer Language Model Built from Scratch

Jan. 2026 – May 2026

University of Notre Dame | Python, PyTorch, pandas, scikit-learn

- Engineered a generative transformer LLM from scratch (tokenization, multi-head attention, backpropagation, and full training pipelines) with RAG-based orchestration to minimize hallucinations and optimize context-window performance
- Implemented foundational AI/ML algorithms end-to-end: uninformed/informed search, logistic regression, SVMs, clustering, PCA, CNNs, RNNs, and reinforcement learning
- Built end-to-end generative AI workflows spanning prompt engineering, retrieval-augmented generation, context optimization, model evaluation, bias analysis, and agent-based task orchestration aligned to safety benchmarks

TECHNICAL SKILLS

Languages: Python, C++, C, Rust, OCaml/OxCaml, SQL, Java, JavaScript, PHP, MATLAB, HTML/CSS, Shell

Quantitative Finance: Market microstructure, limit order books, price-time priority, ETF arbitrage/iNAV, NBBO & TAQ tick data, WRDS, order-book imbalance/micro-price, market-impact modeling (square-root law, Almgren-Chriss), optimal execution (TWAP/VWAP/IS), latency modeling & co-location, AR(1) mean reversion, lead-lag cross-correlation, event studies, Sharpe-ratio optimization, backtesting, CVaR/tail-risk analysis

ML / AI: PyTorch, TensorFlow, Keras, LightGBM, pandas, NumPy, scikit-learn, Qiskit (quantum-ML), Anthropic SDK, OpenAI Codex, Gemini 3.1 Pro; transformers/attention, RAG, fine-tuning, model evaluation, bias analysis, adversarial attack detection

Systems & Security: Linux/Unix, multithreading/concurrency (OCaml multicore domains, lock-free atomics), TCP/UDP multicast, binary wire protocols, GC/allocation profiling, cache-friendly data layouts, Wireshark, safetensors, model serialization security, memory-safe Rust

Engineering & Tools: Git, GitHub Actions CI/CD, DevOps pipelines, property-based testing (qcheck), pytest, PyArrow/Parquet, large-scale data processing, GitHub/REST API integration, Advanced Excel, Agile, React, UX/DevEx

RELEVANT COURSEWORK

Quant / HFT / AI: High-Frequency Trading Technologies (A), Intro to Artificial Intelligence (A), Probability & Statistics (A-), Design/Analysis of Algorithms (A-), Theory of Computing (A), Data Structures, Linear Algebra & Differential Equations, Calculus I–III

Systems & Security: Computer Security (A), Programming Paradigms (A), Modern Web Development (A), Systems Programming, Computer Architecture, Computer Networks, Operating System Principles, Logic Design, Fundamentals of Computing (A), Discrete Mathematics

Business & Economics: Integrated Engineering & Business Fundamentals (A), Advanced Topics: Integrated EG & Business (A-), Idea Discovery Lab (A-), Foundations of Accountancy (A-), Principles of Microeconomics